



# UNIVERSIDAD DE GRANADA

Desarrollo de Software

## Práctica 3 - Diseño, Implementación y Pruebas Unitarias

**Autores:** Jorge García Rubio y Raúl Gutiérrez Manrique

**Repositorio:** [github.com/jrgeegr/Practica3\\_Grupal\\_DS/](https://github.com/jrgeegr/Practica3_Grupal_DS/)

3 de mayo de 2026

# Índice

|                                            |   |
|--------------------------------------------|---|
| 1. Objetivo de la Práctica                 | 2 |
| 2. Arquitectura (Patrones de Diseño)       | 2 |
| 3. Estructura de Archivos.                 | 3 |
| 4. Programa Principal.                     | 3 |
| 5. Detalles Adicionales de Implementación. | 4 |
| 6. Pruebas Unitarias.                      | 5 |

## 1. Objetivo de la Práctica

El objetivo principal es diseñar e implementar un sistema de gestión de servicios turísticos utilizando el lenguaje Dart y el framework Flutter. La aplicación debe permitir la creación de paquetes de viaje personalizados mediante la combinación de distintos servicios (vuelos y hoteles), aplicando de forma dinámica diferentes políticas de precios.

## 2. Arquitectura (Patrones de Diseño)

Para resolver los requisitos de forma extensible y limpia, hemos aplicado dos patrones de diseño:

1. **Patrón Composite.** Se utiliza para tratar a los servicios individuales como a los paquetes de servicios de manera uniforme.
  - Componente (**ServicioTuristico**): interfaz que define el método de calcular el precio.
  - Hoja (**Vuelo y Hotel**): implementaciones individuales que calculan su precio delegando en una política
  - Compuesto (**Paquete**): contiene una lista de **ServicioTuristico**. Su método para calcular el precio es recursivo; suma los precios de todos sus hijos, ya sean hojas u otros paquetes anidados.
2. **Patrón Strategy.** Se aplica para gestionar las variaciones en el cálculo de precios sin modificar las clases de los servicios.
  - Interfaz de Estrategia: **PoliticaVuelo** y **PoliticaHotel**.
  - Estrategias Concretas: **TarifaLowCost**, **TarifaBusiness**, **SoloAlojamiento** y **TodoIncluido**.

Esto permite que un Vuelo u Hotel cambie su comportamiento de tarificación en tiempo de ejecución simplemente intercambiando su objeto de política.

El diagrama UML del ejercicio es el siguiente:

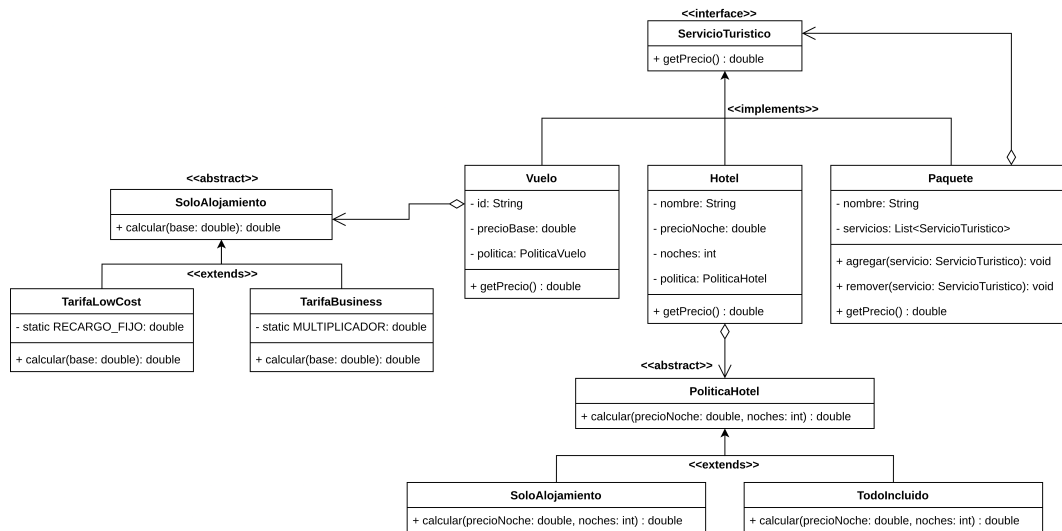


Figura 1: Diagrama de clases UML.

### 3. Estructura de Archivos.

Siguiendo las buenas prácticas de Flutter, hemos organizado el código para separar la interfaz de la lógica:

- `lib/`: Directorio principal de código fuente.
  - `servicio_turistico.dart`: Interfaz base del patrón Composite.
  - `vuelo.dart` / `hotel.dart`: Componentes hoja.
  - `paquete.dart`: Clase compuesta con métodos de gestión (`agregar`, `remover`).
  - `politica_vuelo.dart` / `politica_hotel.dart`: Definición de las estrategias de precio.
  - `main.dart`: Punto de entrada de la aplicación y definición de la UI.
- `test/`:
  - `reserva_test.dart`: Batería de 12 pruebas unitarias organizadas por grupos.

### 4. Programa Principal.

El archivo `main.dart` gestiona el ciclo de vida de la aplicación mediante un `StatefulWidget`.

1. **Estado:** Mantiene un `paqueteRaiz` (donde se construye el viaje) y un `inventarioDisponible` (servicios precargados).
2. **Interfaz:** Se divide la pantalla en dos columnas: a la izquierda el catálogo y a la derecha el carrito/paquete.
3. **Reactividad:** Al modificar una política en un `DropDownButton` o añadir/quitar un servicio, se invoca `setState()`. Esto provoca que el *widget* se reconstruya y llame de nuevo a `paqueteRaiz.getPrecio()`, actualizando el total instantáneamente.

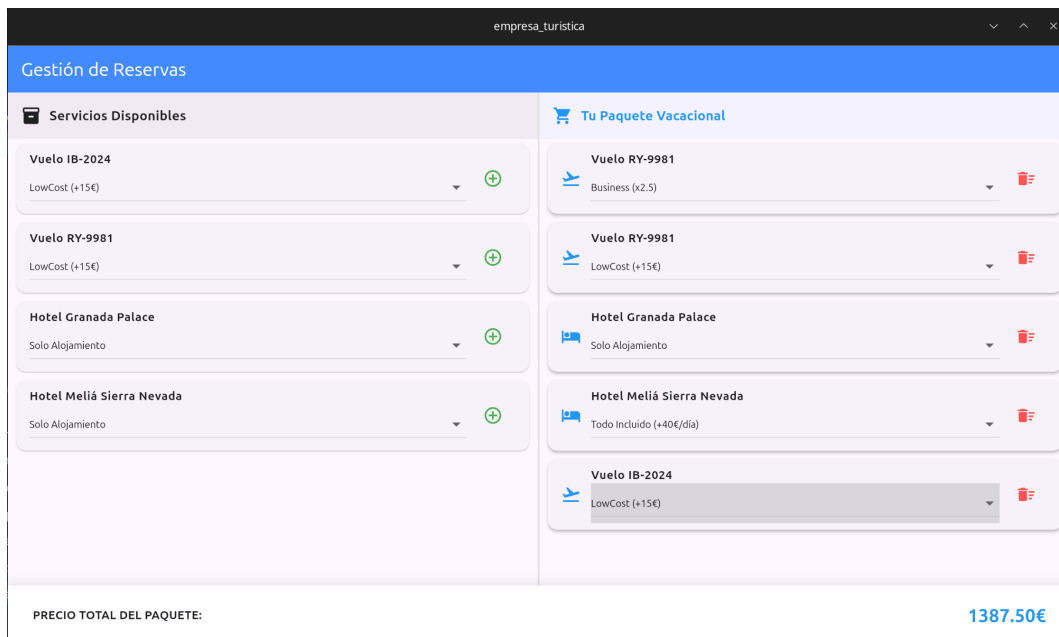


Figura 2: Programa Principal.

## 5. Detalles Adicionales de Implementación.

### Sobrecarga de Operadores (`==` y `hashCode`)

Para que los selectores desplegables (`DropDownButton`) de Flutter funcionen correctamente sin usar el patrón Singleton, hemos sobrescrito los operadores de igualdad en las clases de política.

- **Por qué:** Flutter necesita comparar la política asignada al objeto con las opciones de la lista. Por defecto, Dart compara referencias de memoria. Al sobrescribir `==`, permitimos que dos instancias distintas de `TarifaLowCost` sean consideradas iguales, evitando errores de “valor no encontrado” en la interfaz.

## Clonación de instancias

Durante el desarrollo, se detectó que al añadir múltiples veces el mismo servicio desde el inventario al paquete, los cambios en las políticas se replicaban en todas las copias. Esto ocurría porque se compartía la misma referencia de memoria del objeto original.

Para ello se ha implementado un método `copy()` (basado en el patrón prototype) en las clases `Vuelo` y `Hotel`. Así, al pulsar el botón de añadir en la interfaz, el sistema no entrega el objeto original del inventario, sino que invoca `servicio.copy()`.

## Configuración del Proyecto (`pubspec.yaml`)

Es el archivo de gestión de dependencias. Para esta práctica, los puntos clave fueron:

- **dependencies:** Inclusión de `cupertino_icons` para la estética.
- **dev\_dependencies:** Inclusión del paquete `test` para habilitar el entorno de pruebas unitarias. Sin esta declaración, el compilador no reconoce los comandos de prueba ni las importaciones de `package:test/test.dart`.

## 6. Pruebas Unitarias.

Para garantizar la integridad del sistema, se ha implementado una batería de 12 tests unitarios utilizando el paquete `test` de Dart.

- Grupo Políticas de Tarifación:

```
11 group('Grupo Políticas de Tarifación', () {
12   test('TarifaLowCost añade su recargo de gestión constante (15.0) al precio base', () {
13     final politica = TarifaLowCost();
14     expect(politica.calcular(10.0), equals(25.0));
15   });
16
17   test('TarifaBusiness aplica su multiplicador interno (2.5) correctamente', () {
18     final politica = TarifaBusiness();
19     expect(politica.calcular(10.0), equals(25.0));
20   });
21
22   test('RegimenSoloAlojamiento calcula el producto de noches por precio', () {
23     final politica = SoloAlojamiento();
24     expect(politica.calcular(100.0, 3), equals(300.0));
25   });
26
27   test('RegimenTodoIncluido incorpora su suplemento diario (40.0) al coste', () {
28     final politica = TodoIncluido();
29     expect(politica.calcular(100.0, 2), equals(280.0));
30   });
31 });
```

Figura 3: Test unitarios, Grupo 1.

- Grupo Servicios Individuales (Hojas):

```

33 group('Grupo Servicios Individuales (Hojas)', () {
34   test('El constructor de Vuelo lanza excepción ante precio base negativo', () {
35     expect(() => Vuelo("V-1", -10.0, TarifaLowCost()), throwsArgumentError);
36   });
37
38   test('El constructor de Hotel impide estancias con cero o menos noches', () {
39     expect(() => Hotel("H-1", 50.0, 0, SoloAlojamiento()), throwsArgumentError);
40   });
41
42   test('El método getPrecio() de Vuelo delega el cálculo en su política', () {
43     final vuelo = Vuelo("V-1", 100.0, TarifaBusiness());
44     expect(vuelo.getPrecio(), equals(250.0));
45   });
46
47   test('El método getPrecio() de Hotel retorna el valor procesado por su régimen', () {
48     final hotel = Hotel("H-1", 100.0, 1, TodoIncluido());
49     expect(hotel.getPrecio(), equals(140.0));
50   });
51 });

```

Figura 4: Test unitarios, Grupo 2.

- Grupo Agrupación de Paquetes:

```

53 group('Grupo Agrupación de Paquetes', () {
54   test('Un Paquete recién instanciado sin servicios devuelve cero', () {
55     final paquete = Paquete("Vacio");
56     expect(paquete.getPrecio(), equals(0.0));
57   });
58
59   test('Paquete realiza la suma aritmética de todos sus componentes directos', () {
60     final paquete = Paquete("Mix");
61     paquete.agregar(Vuelo("V-1", 100.0, TarifaLowCost())); // 115
62     paquete.agregar(Hotel("H-1", 100.0, 1, SoloAlojamiento())); // 100
63     expect(paquete.getPrecio(), equals(215.0));
64   });
65
66   test('La estructura recursiva permite sumar paquetes anidados correctamente', () {
67     final paqueteRaiz = Paquete("Raiz");
68     final subPaquete = Paquete("Sub");
69
70     subPaquete.agregar(Vuelo("V-1", 100.0, TarifaLowCost())); // 115
71     paqueteRaiz.agregar(subPaquete);
72     paqueteRaiz.agregar(Hotel("H-1", 100.0, 1, SoloAlojamiento())); // 100
73
74     expect(paqueteRaiz.getPrecio(), equals(215.0));
75   });
76
77   test('Cambiar la política de un servicio existente actualiza el total del paquete raiz', () {
78     final paqueteRaiz = Paquete("Raiz");
79     final vuelo = Vuelo("V-1", 100.0, TarifaBusiness()); // 250
80     paqueteRaiz.agregar(vuelo);
81
82     expect(paqueteRaiz.getPrecio(), equals(250.0));
83
84     // Cambio dinámico de política (Strategy)
85     vuelo.politica = TarifaLowCost(); // Ahora 115
86     expect(paqueteRaiz.getPrecio(), equals(115.0));
87   });
88 });
89 }

```

Figura 5: Test unitarios, Grupo 3.